INSTITUTO TECNOLÓGICO SUPERIOR DE LA
REGIÓN DE LOS LLANOS

INGENIERÍA MECATRÓNICA

Ciberfisica

Actividad: U1A5. (20%) DASHBOARD/NOTEBOOK

Nombre: Angel Daniel Cisneros Perez

Docente: Osbaldo Aragón Banderas

Fecha: 2026-02-24

TECNM
TECNOLÓGICO NACIONAL DE
MÉXICO

Índice

1. Introducción..... 1
2. Objetivo..... 1
3. Descripción del sistema (extremo a extremo)..... 2
4. Adquisición y telemetría por Serial (payload y trazabilidad temporal)..... 2
4.1 Microcontrolador y ADC..... 2
4.2 Payload compacto (obligatorio)..... 3
4.3 Frecuencia de actualización y trazabilidad..... 3
5. Notebook de calibración en Python..... 5
5.1 Componentes mínimos implementados..... 5
6. Procedimiento de calibración..... 7
6.1 Captura NEGRO (línea)..... 7
6.2 Captura BLANCO (fondo)..... 8
6.3 Cálculo automático posterior a la calibración..... 8
7. Verificación de funcionalidad (Pruebas 1–3)..... 9
Prueba 1 — Contraste (obligatoria)..... 9
Prueba 2 — Repetibilidad (obligatoria)..... 10
Prueba 3 — Detección por umbral (obligatoria)..... 11
8. Registro CSV y evidencias..... 13
8.1 Requisitos del registro..... 13
8.2 Evidencia requerida..... 13
9. Justificación técnica del entorno (JupyterLab vs Notebook en VS Code)..... 14
10. Explicación técnica..... 15

1. Introducción

Los sensores analógicos de reflectancia (IR) en un seguidor de línea convierten diferencias de reflectividad entre **línea (negro)** y **fondo (blanco)** en una señal analógica que se digitaliza mediante el ADC del microcontrolador. Sin una calibración previa, la lectura cruda puede variar significativamente por iluminación ambiente, distancia al piso, inclinación del sensor y tolerancias del emisor/receptor IR. Esto provoca que un umbral fijo no sea confiable y se presenten falsas detecciones de línea. En esta práctica se implementó un notebook operativo para visualizar lecturas en tiempo casi real, capturar referencias de calibración (NEGRO/BLANCO), calcular umbrales/normalización, validar funcionamiento mediante pruebas y registrar telemetría en un CSV verificable.

2. Objetivo

Desarrollar un **notebook operativo en Python** que permita:

- Visualizar lecturas analógicas (mínimo 2 canales) en tiempo casi real.
- Calibrar por canal mediante capturas **NEGRO** y **BLANCO**.
- Calcular automáticamente **normalización** y **umbral** y mostrarlos en pantalla.
- Verificar funcionamiento del sensor mediante **contraste, repetibilidad y detección**.
- Registrar un **CSV con ≥300 muestras** (incluyendo `t_ms`, canales, y parámetros asociados).

3. Descripción del sistema (extremo a extremo)

El sistema implementado funciona en cadena completa:

Sensores analógicos IR (s1, s2) → ESP32 (ADC 12-bit) → USB/Serial → PC → JupyterLab (Python Notebook) → Dashboard (gráficas + UI) → Cálculo de calibración (min/max/thr/norm) → Indicadores de detección → Registro CSV

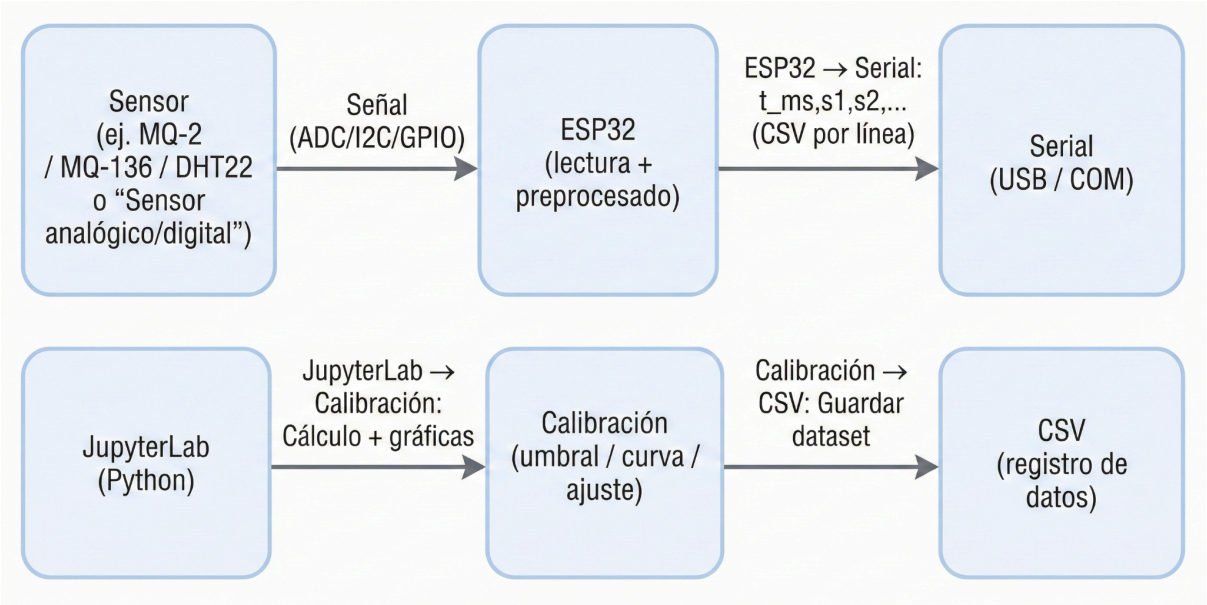

Fig 1. Diagrama extremo a extremo sensor→ESP32→Serial→JupyterLab→calibración→CSV.

4. Adquisición y telemetría por Serial (payload y trazabilidad temporal)

4.1 Microcontrolador y ADC

Se utilizó un **ESP32** leyendo dos canales analógicos (s1 y s2) con resolución ADC de **12 bits**, por lo que el rango crudo esperado es **0 a 4095**. Para asegurar una

lectura útil dentro del rango de voltaje, se configuró la atenuación adecuada en pines ADC1 (recomendado para estabilidad).

4.2 Payload compacto (obligatorio)

Se definió un payload **CSV** por su baja sobrecarga y parseo directo en Python:

`(t_ms,s1,s2)`

- **`t_ms`:** timestamp en milisegundos **(`millis()`)**, garantiza orden temporal.

- **`s1`, `s2`**: lecturas ADC crudas por canal.

Ejemplo:
`1833540,68,34`

4.3 Frecuencia de actualización y trazabilidad

- El muestreo se realizó con intervalo estable (ej. ~100 ms ≈ 10 Hz), cumpliendo el requisito de **≥2 Hz**.

- El notebook mantuvo un contador de muestras y mostró la última marca temporal **`t_ms`** para asegurar que la recepción es continua y ordenada.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

* Executing task: C:\Users\Angel Daniel\.platformio\penv\Scripts\platformio.exe device monitor
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4
240,4095,4095
340,4095,4085
440,4095,4095
540,4095,4095
640,4095,4095
740,4095,4087
840,4095,4095
940,4095,4095
1040,4095,4095
1140,4095,4095
1240,4095,4095
1340,4095,4095
1440,4095,4095
1540,4095,4095
1640,4095,4095
1740,4095,4095
1840,4095,4095
1940,4095,4095
2040,4095,4091
2140,4095,4095
2240,4095,4095
2340,4095,4095
2440,4095,4095
2540,4095,4095
2640,4095,4079
Default (U1A5_ESP32_Serial)    Auto
```

Fig 2. Evidencia de salida serial (t_ms,s1,s2) en recepción.

5. Notebook de calibración en Python

5.1 Componentes mínimos implementados

El notebook implementó los siguientes elementos obligatorios:

Gráficas en tiempo casi real (N≥100):

- Gráfica **RAW**: **`s1 y s2`** (últimos N datos).
- Gráfica **NORM**: **`s1_norm y s2_norm`** en escala 0–1.

Indicadores numéricos:

- Valor actual por canal (última muestra).
- Contador de muestras recibidas.
- Estimación de frecuencia (Hz aproximados).

Controles de calibración (UI):

- Botón **Capturar NEGRO**: guarda **`min_line`** por canal.
- Botón **Capturar BLANCO**: guarda **`max_floor`** por canal.
- Botón de **Reset** (reinicia perfil de calibración).

Parámetros mostrados en pantalla:

- **`min_line`**, **`max_floor`**, **`thr`**, Δ repetibilidad y estado LINEA/NO.

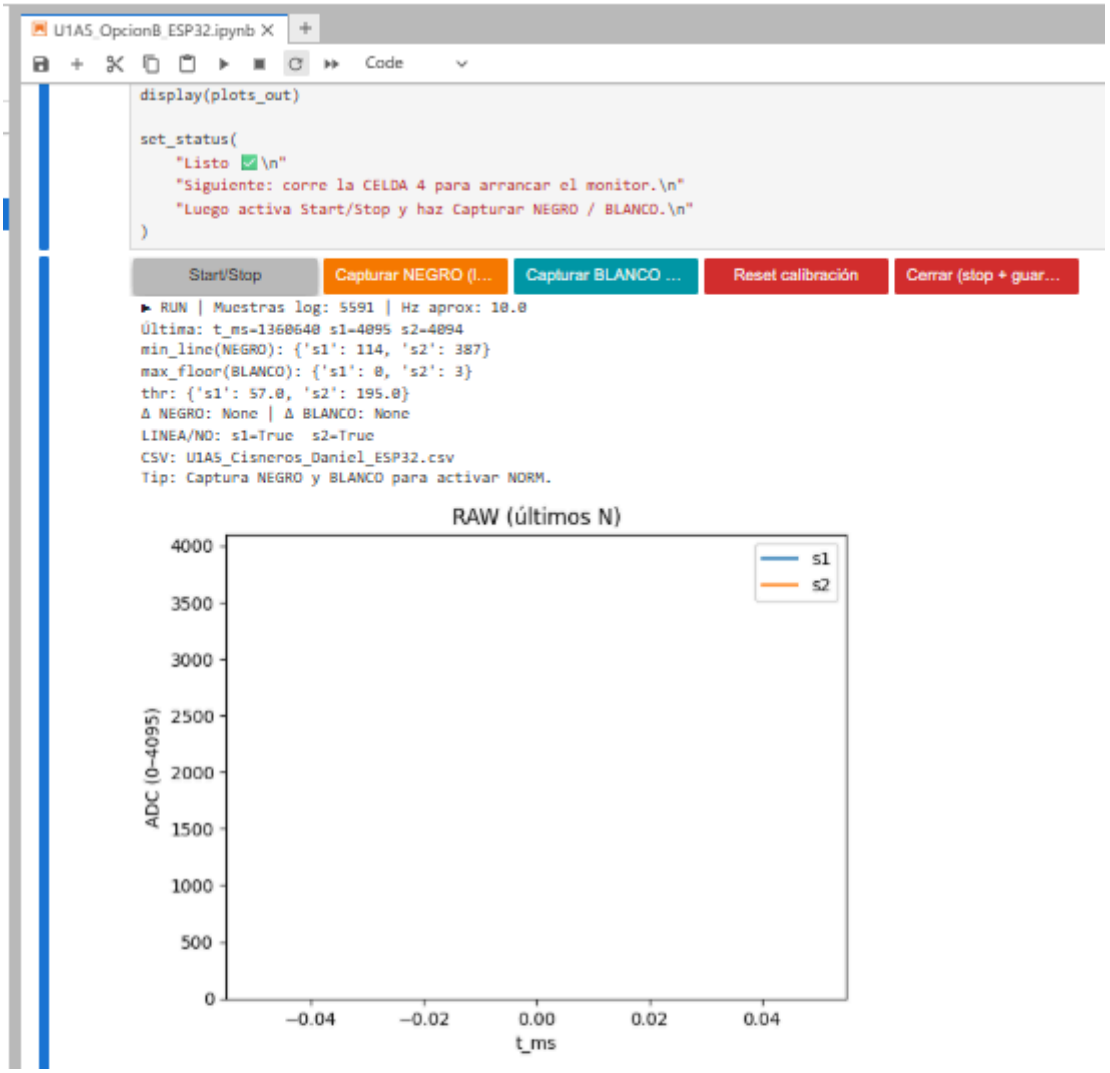

Fig 3. Notebook en JupyterLab mostrando controles y gráfica RAW.

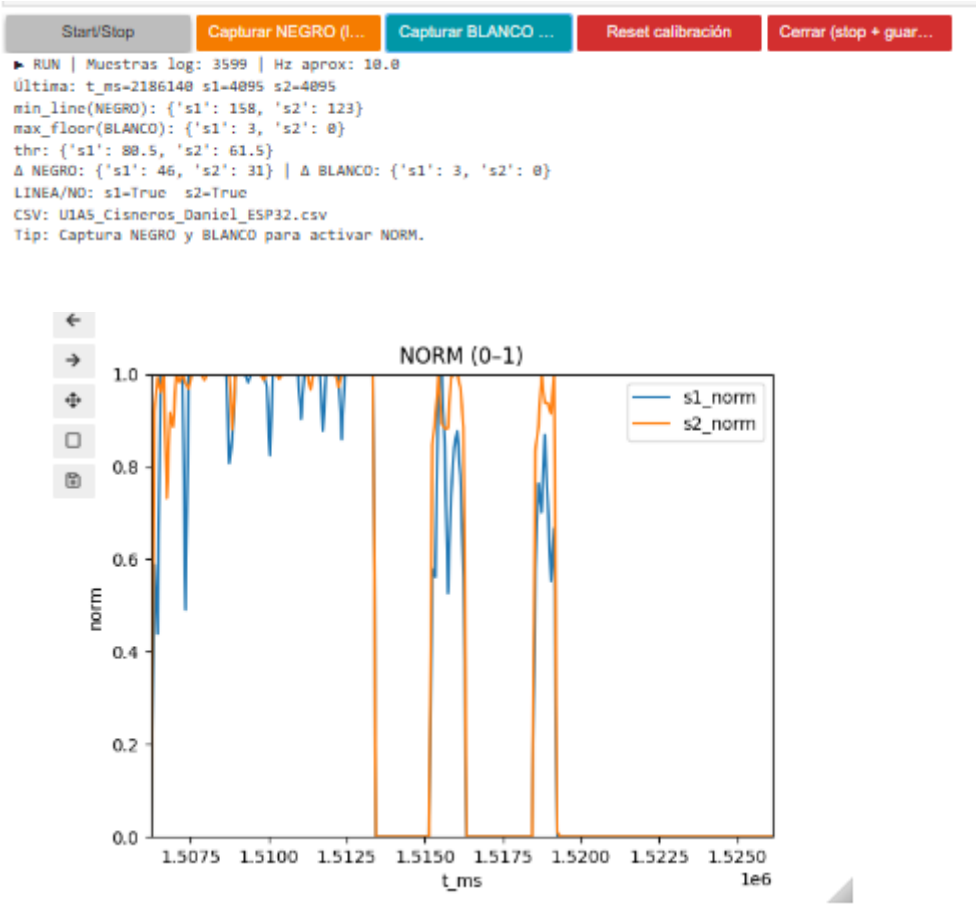

Fig 4. Notebook mostrando gráfica NORM y parámetros calculados (min/max/thr).

6. Procedimiento de calibración

La calibración se realizó por canal, ya que cada sensor puede tener diferente offset/ganancia por montaje.

6.1 Captura NEGRO (línea)

1. Colocar los sensores sobre la línea negra.

2. Esperar estabilización breve (1–2 s).

3. Presionar **Capturar NEGRO**.

4. Se registran:
   - **`min_line_s1 = s1_actual`**
   - **`min_line_s2 = s2_actual`**

6.2 Captura BLANCO (fondo)

1. Colocar los sensores sobre el fondo blanco.
2. Esperar una breve estabilización.
3. Presionar **Capturar BLANCO**.
4. Se registran:
   - **`max_floor_s1 = s1_actual`**
   - **`max_floor_s2 = s2_actual`**

6.3 Cálculo automático posterior a la calibración

Con **`min_line` y `max_floor`,** el notebook calcula:

Umbral sugerido por canal (punto medio):

$$thr = \frac{min_line + max_floor}{2}$$

Normalización (limitada 0–1):

$$norm = \frac{s - min_line}{max_floor - min_line}$$

y se limita a:

$$norm \in [0, 1]$$

para evitar valores fuera de rango por ruido o saturación.

7. Verificación de funcionalidad (Pruebas 1–3)

Prueba 1 — Contraste (obligatoria)

Se colocó el sensor en **blanco** y luego en **negro**, verificando:

- Cambio claro en **gráfica RAW**.
- Cambio inmediato en el valor actual mostrado.
- Tras calibración, cambio coherente en **NORM** (cerca de 1 en blanco y cerca de 0 en negro, según calibración).

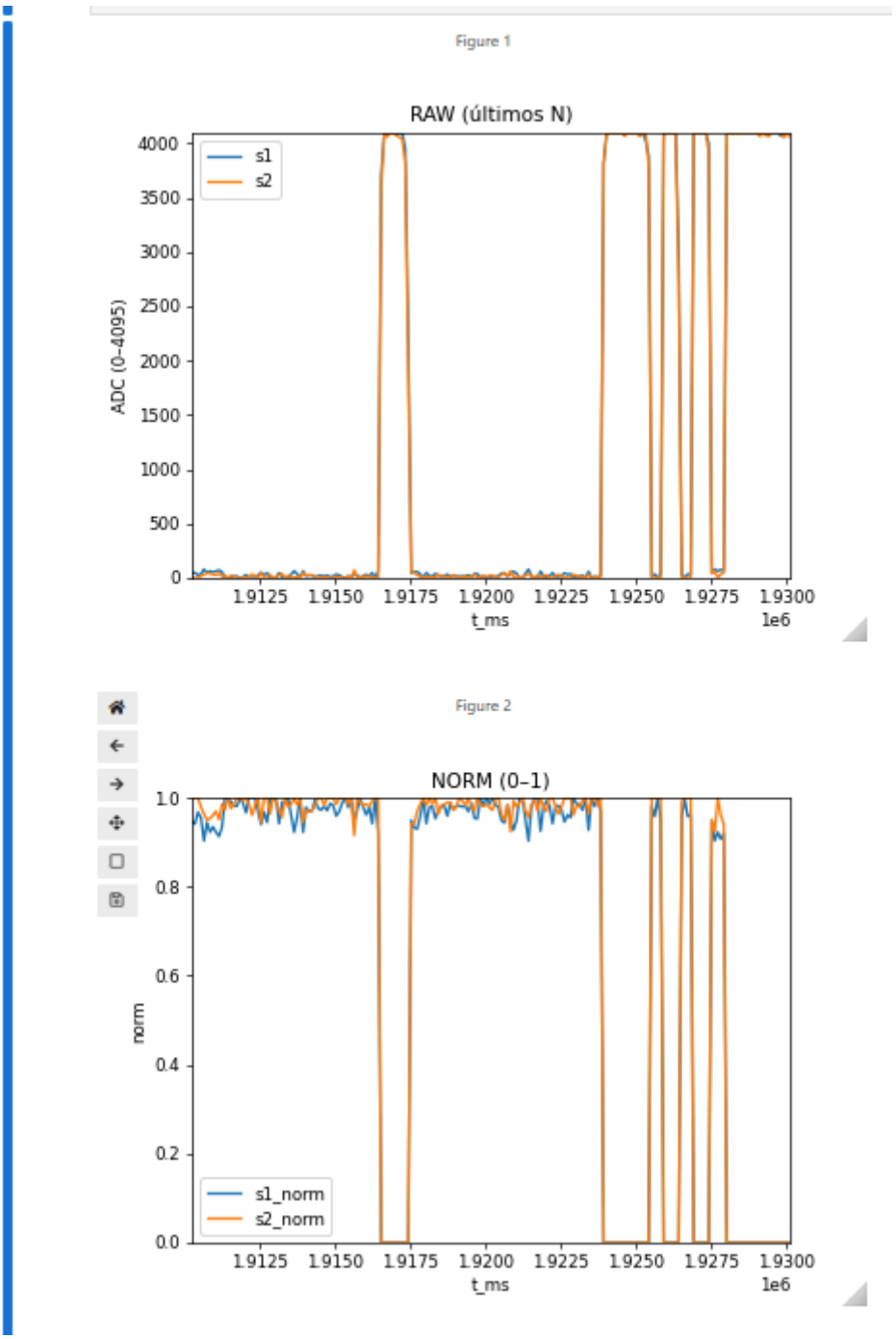

Fig 5. Evidencia de contraste: blanco vs negro en RAW (antes/después).

Prueba 2 — Repetibilidad (obligatoria)

Se repitieron capturas dos veces en cada condición:

- NEGRO: Capturar NEGRO (1) y Capturar NEGRO (2)
- BLANCO: Capturar BLANCO (1) y Capturar BLANCO (2)

Se reportó diferencia por canal:

Δ=∣captura2−captura1∣\Delta = |captura_2 - captura_1|Δ=∣captura2−captura1∣

Resultados:

- Δ NEGRO: Δs1 = 158 ; Δs2 = 123
- Δ BLANCO: Δs1 = 3 ; Δs2 = 0

Interpretación: Δ bajo indica buena repetibilidad; Δ alto sugiere variación por iluminación/altura/movimiento y puede requerir calibración o ajuste físico.

```
min_line(NEGRO): {'s1': 158, 's2': 123}
max_floor(BLANCO): {'s1': 3, 's2': 0}
```

Fig 6. Evidencia de repetibilidad mostrando Δ calculado por canal.

Prueba 3 — Detección por umbral (obligatoria)

Con `thr` calculado, el notebook generó un indicador booleano:

- **LINEA** si la lectura cae del lado correspondiente a negro respecto a `thr`.

- **NO LÍNEA** en caso contrario.

Se verificó que al alternar superficie:

- En negro → se muestre **LINEA**
- En blanco → se muestre **NO LINEA**

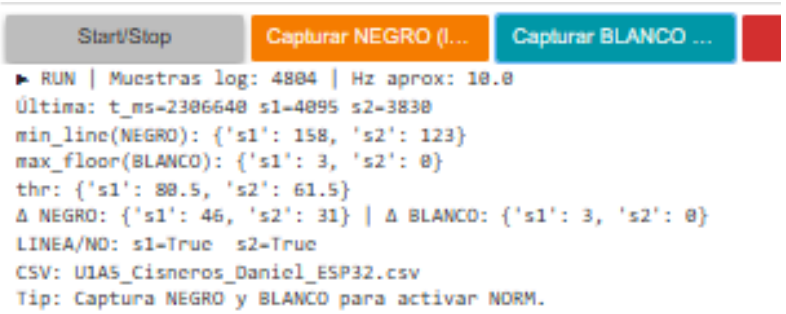

Fig 7. Evidencia del indicador “LINEA/NO LÍNEA” por canal funcionando.

8. Registro CSV y evidencias

8.1 Requisitos del registro

Se registró un CSV con recepción continua **≥60 s**, generando ≥300 muestras (a 10 Hz se obtienen aproximadamente 600).

Columnas incluidas:

- `t_ms`
- `s1`, `s2`

  `s1_norm`, `s2_norm` (opcional recomendado)
- `thr1`, `thr2`
- `line1`, `line2`

8.2 Evidencia requerida

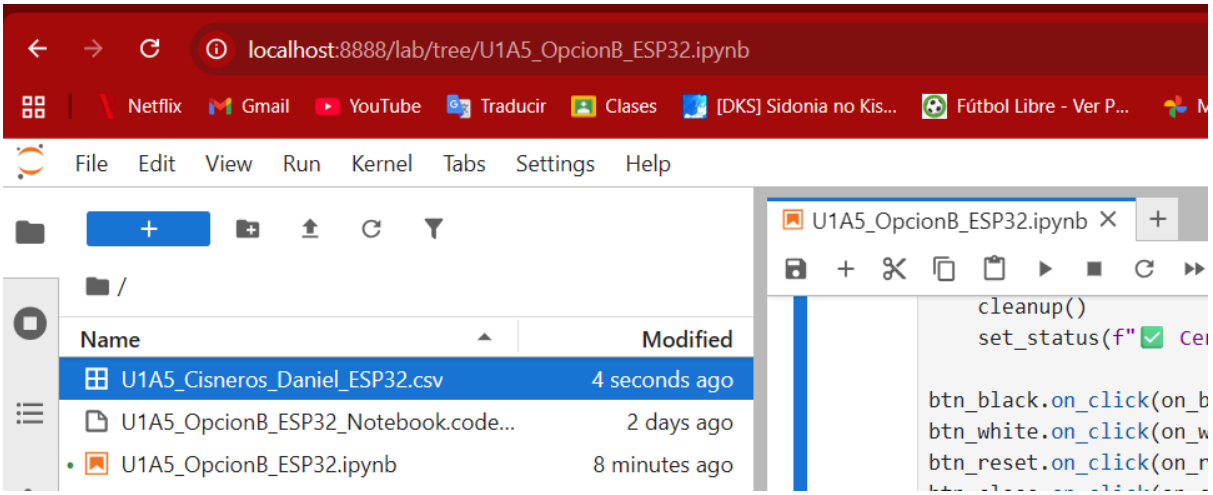

Fig 8. Evidencia de archivo CSV generado y conteo total de muestras.

Fragmentos de CSV

	A	B	C	D	E	F	G	H	I
1	t_ms	s1	s2	s1_norm	s2_norm	thr1	thr2	line1	line2
2	1826340	4095	4095						
3	1826440	4095	4095						
4	1826540	4095	4091						
5	1826640	4095	4069						
6	1826740	4095	4095						
7	1826840	4095	4095						
8	1826940	4095	4095						
9	1827040	4095	4054						
10	1827140	4095	4095						

6354	2461540	4095	4095	0	0	80.5	61.5	1	1
6355	2461640	4095	4095	0	0	80.5	61.5	1	1
6356	2461740	4095	4095	0	0	80.5	61.5	1	1
6357	2461840	4095	4095	0	0	80.5	61.5	1	1
6358	2461940	4095	4094	0	0	80.5	61.5	1	1
6359	2462040	4095	4095	0	0	80.5	61.5	1	1
6360	2462140	4095	4095	0	0	80.5	61.5	1	1
6361	2462240	4095	4095	0	0	80.5	61.5	1	1
6362	2462340	4095	4095	0	0	80.5	61.5	1	1
6363	2462440	4095	4095	0	0	80.5	61.5	1	1
6364	2462540	4095	4095	0	0	80.5	61.5	1	1

U1A5_Cisneros_Daniel_ESP32

Conteo total de muestras: 6364 (≥300)

9. Justificación técnica del entorno (JupyterLab vs Notebook en VS Code)

Durante el desarrollo se intentó ejecutar el notebook en el entorno de Jupyter integrado en Visual Studio Code. Sin embargo, se presentó un problema de compatibilidad de **widgets y backends interactivos de Matplotlib**: el backend `ipympl` requiere que el frontend registre correctamente el módulo JavaScript `jupyter-matplotlib` para renderizar figuras actualizables en tiempo real. En VS Code se observaron salidas en blanco y errores relacionados con el registro del

módulo, lo cual impidió actualizar las gráficas de forma estable y mantener una UI funcional sin duplicación o bloqueo.

Se optó por **JupyterLab en navegador (localhost)** porque:

- Implementa un frontend Jupyter completo con soporte robusto para `ipywidgets` e `ipympl`.
- Permite actualizar figuras interactivas en tiempo real sin que el documento se "desplace" o genere salidas inconsistentes.
- Funciona **offline** (local), manteniendo acceso al puerto serial del ESP32, lo que es indispensable para telemetría USB/Serial.

En síntesis, **JupyterLab fue seleccionado como entorno operativo** para garantizar estabilidad del dashboard, compatibilidad con widgets y visualización en tiempo real, cumpliendo los requisitos de la actividad.

10. Explicación técnica

Se calibraron sensores analógicos de reflectancia conectados a un ESP32 usando su ADC de 12 bits (0–4095) para obtener lecturas confiables de línea y fondo. La calibración es necesaria porque el valor crudo depende de iluminación, distancia al piso y variaciones de montaje, haciendo que un umbral fijo sea inestable. Por ello se capturaron referencias por canal: `min_line` en NEGRO y `max_floor` en BLANCO, y se calculó un umbral `thr` como punto medio y una normalización `norm` en 0–1 para estandarizar lecturas. El orden temporal se garantizó enviando `t_ms` en cada muestra por Serial, permitiendo trazabilidad y conteo continuo. Se consideró que el sensor funciona si: (1) existe contraste claro blanco vs negro en la gráfica y el valor actual, (2) la repetibilidad es aceptable al reportar Δ en capturas repetidas, y (3) la detección booleana LINEA/NO LINEA cambia coherentemente usando `thr`. Finalmente, se registró un CSV con ≥300 muestras verificables como evidencia del pipeline extremo a extremo.

11. Conclusiones

El notebook en Python permitió implementar una calibración operativa de sensores analógicos de un seguidor de línea con visualización en tiempo casi real, captura de referencias NEGRO/BLANCO y cálculo automático de umbral y normalización. Las pruebas de contraste, repetibilidad y detección confirmaron la respuesta del sensor y la utilidad del umbral calculado. El registro en CSV con ≥300 muestras proporciona evidencia verificable y base para reutilizar parámetros como perfil de calibración en el control del robot. La elección de JupyterLab como entorno final aseguró compatibilidad con gráficos interactivos y estabilidad en la operación del dashboard serial.